

# TrailGuard: Off-Grid Safety & Tracking System

## Project Summary for Presentation

### 1. Project Mission & Core Problem

- **Mission:** TrailGuard is an off-grid safety, tracking, and communication system designed for hikers, park rangers, and outdoor enthusiasts operating in environments with zero cellular or internet connectivity.
- **The Problem:** Traditional satellite communication requires expensive subscriptions and line-of-sight. Deep wilderness trails lack cellular networks, making emergency communication difficult.
- **The Solution:** A decentralized LoRa (Long Range) Radio Mesh Network powered by the open-source Meshtastic protocol, creating a localized, free-to-use communication web across trails.

### 2. High-Level Concept

- **Hikers:** Carry a small, battery-powered LoRa radio device ("Personal Node") connected to their smartphone via Bluetooth. They use the **TrailGuard Mobile App**.
- **Rangers:** Deploy static LoRa "Relay Nodes" throughout the forest to form a robust radio mesh network.
- **Command Center:** Uses a Gateway Node to receive mesh traffic. The data feeds into the **TrailGuard Dashboard**, allowing Rangers to monitor hikers in real-time and receive SOS alerts.

### 3. Key Features

- **Offline Functionality:** The mobile app works fully offline with pre-downloaded vector maps (MBTiles).
- **Real-time SOS & Check-ins:** Hikers can broadcast their location or trigger high-priority SOS alerts.
- **Decentralized Mesh Networking:** Messages bounce from node to node to cover long distances over mountains.
- **Centralized Dashboard:** A comprehensive web interface for Rangers to view live maps, hiker statuses, and security logs.
- **WiFi Simulator (Demo Mode):** A built-in simulator allowing testing over a local network without physical LoRa hardware.

### 4. Architecture & Levels

The system operates on three distinct levels:

1. **Level 1: The Edge / Client (Mobile App):** The user interface for the hiker. Handles offline mapping and cryptographic signing.
2. **Level 2: The Transport Mesh (Hardware Nodes):** ESP32-based LoRa boards running custom Meshtastic firmware. Handles routing, signature verification, and replay protection.
3. **Level 3: The Command Center (Dashboard):** Data ingestion, persistence (SQLite), and the Ranger UI (Python/Flask).

### 5. Protocols & Security

TrailGuard operates in a zero-trust environment where anyone can listen to the frequencies. Security is paramount.

- **Ed25519 Cryptography:** Every hiker generates a unique Public/Private keypair on their phone. All messages are signed with the Private Key (stored securely) and verified by the network, preventing spoofing.
- **Replay Protection:** To prevent attackers from rebroadcasting old SOS signals, the firmware uses timestamps and a bounded memory Hash Ring (djb2) to drop duplicate messages before they reach the Dashboard.

- **Compact Binary Payloads:** Due to low LoRa bandwidth, data is packed into extremely dense binary arrays (e.g., Signature + PubKey + Timestamp + Lat + Lng + Type).

## 6. Tech Stack & Libraries

- **Mobile App (Hiker Client):** React Native (Expo), `react-native-maplibre-gl` for offline mapping, `react-native-quick-crypto` for fast Ed25519 cryptography, and `react-native-keychain` for secure key storage.
- **Firmware (Hardware Nodes):** C++ compiled via PlatformIO for ESP32 (Heltec boards). Custom module built on top of Meshtastic using `TweetNaCl` for cryptography.
- **Command Center (Dashboard):** Python 3 with Flask, SQLite3 for a local serverless database, Vanilla HTML/CSS/JS for the frontend UI, and the `meshtastic` Python library for hardware bridging.

## 7. Data Flow Overview

1. **Hiker to App:** Hiker triggers a Check-In or SOS on the app.
2. **App to Node (BLE):** Payload is cryptographically signed and sent via Bluetooth to the personal LoRa node.
3. **Node to Mesh (LoRa):** Node broadcasts the payload over 915MHz LoRa. Relay nodes bounce it through the forest.
4. **Mesh to Gateway (USB):** The Ranger Station's Gateway node receives the signal and passes it via USB to the computer.
5. **Gateway to Backend (Python):** Python backend verifies the signature, prevents replays, and saves it to SQLite.
6. **Backend to Dashboard (HTTP):** The web dashboard polls the backend and instantly updates the UI map and security alerts.